

MIIA0106

PYTHON AND C PROGRAMMING LANGUAGE



Pointer ใน C++

อาจารย์ สุทิศ องอาจ

ครั้งที่ 7



WWW.MUT.AC.TH

ตารางสอนรายวิชา **MIIA0106**

Section A

วัน	คาบ	เวลาเรียน	ห้อง
อังคาร (TUE)	คาบ 2	12.30 – 15.00 น.	D604
อังคาร (TUE)	คาบ 3	15.30 – 18.00 น.	MII203

Section B

วัน	คาบ	เวลาเรียน	ห้อง
เสาร์ (SAT)	คาบ 4	16.00 – 18.30 น.	MII207B
อาทิตย์ (SUN)	คาบ 4	15.30 – 17.30 น.	MII201

สอบประจำภาค วันอาทิตย์ ที่ 15 มีนาคม พ.ศ. 2569
เช้า 09:00-12:00 น.

ขาดสอบ จะได้เป็น FE

1. จินตนาการภาพ "หน่วยความจำ" (Memory)

ก่อนจะเข้าใจ Pointer ให้จินตนาการว่า RAM ของคอมพิวเตอร์คือ "ตู้ล็อกเกอร์" ที่เรียงกันมหาศาล

- **Variable (ตัวแปร):** ชื่อที่เราแปะหน้าตู้ (เช่น score)
- **Value (ค่าข้อมูล):** ของที่เก็บในตู้ (เช่น 100)
- **Address (ที่อยู่):** **หมายเลขตู้** (เช่น 0x001) นี่แหละคือสิ่งที่ Pointer สนใจ!



2. Pointer คืออะไร?



คำนิยาม

Pointer คือ "ตัวแปรชนิดพิเศษ" ที่
ไม่ได้เก็บข้อมูลทั่วไป (เช่น ตัวเลข
หรือ ตัวอักษร) แต่เก็บ **Address** หรือ
"ตำแหน่งที่อยู่" ของตัวแปรอื่น



หน้าที่หลัก

ใช้ "ชี้" (Point) ไปยังตำแหน่งใน
หน่วยความจำ เพื่อให้เราสามารถ
เข้าถึง หรือแก้ไขข้อมูลที่ตำแหน่งนั้น
ได้โดยตรง



ความสัมพันธ์

ถ้าตัวแปร x คือบ้าน
Pointer ptr ก็คือ "กระดาศที่จด
พิกัด GPS" ของบ้านหลังนั้น

3. สัญลักษณ์สำคัญ (Syntax)

& Address-of Operator

แปลว่า: "ขอที่อยู่หน่อย"

```
int x = 10;  
cout << &x;  
// ผลลัพธ์: 0x7ffd... (ตำแหน่งของ x)
```

* Dereference Operator

แปลว่า: "ไปเอาของที่อยู่ในที่อยู่นั้นมา"

```
int* ptr = &x;  
cout << *ptr;  
// ผลลัพธ์: 10 (ค่าของ x)
```

4. ตัวอย่างการทำงานจริง (Visual Code)

```
int score = 100;
int* ptr = &score;

// การแสดงผล
cout << score; // 100
cout << &score; // 0x10A (สมมติ)
cout << ptr; // 0x10A
cout << *ptr; // 100
```

สังเกตว่า ptr เก็บค่า 0x10A ซึ่งเป็นที่อยู่ของ score ดังนั้นเมื่อใช้ *ptr จึงได้ค่า 100 ออกมา

Address	Content	Name	Type	Value
90000000	00	sum	int (4 bytes)	000000FF (255 ₁₀)
90000001	00			
90000002	00			
90000003	FF			
90000004	FF	age	short (2 bytes)	FFFF (-1 ₁₀)
90000005	FF			
90000006	1F	average	double (8 bytes)	1FFFFFFFFFFFFFFFFF (4.45015E-308 ₁₀)
90000007	FF			
90000008	FF			
90000009	FF			
9000000A	FF			
9000000B	FF			
9000000C	FF			
9000000D	FF			
9000000E	90	ptrSum	int* (4 bytes)	90000000
9000000F	00			
90000010	00			
90000011	00			

5. ชนิดตัวแปรกับ Pointer

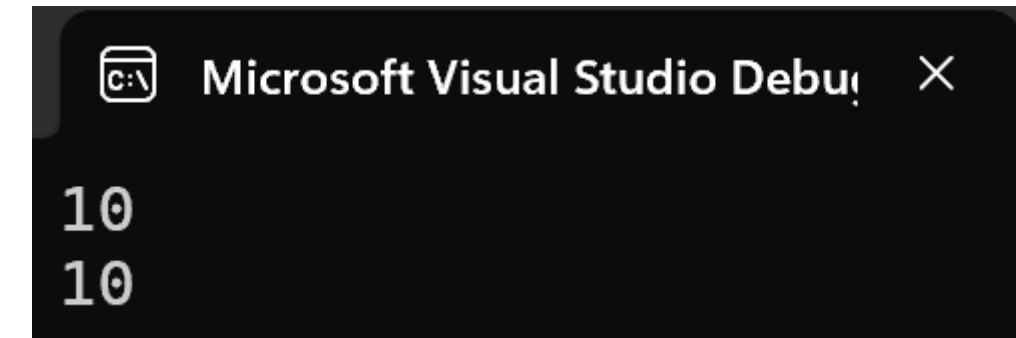
Pointer ต้องประกาศชนิด (Type) ให้ตรงกับตัวแปรที่มันชี้ เพราะคอมพิวเตอร์ต้องรู้ว่าต้องอ่านข้อมูลกี่ Byte

Type	ขนาดข้อมูล (Size)	ตัวอย่าง Pointer	การอ่านค่า
int	4 Bytes	int* p	อ่านทีละ 4 ช่อง
char	1 Byte	char* p	อ่านทีละ 1 ช่อง
double	8 Bytes	double* p	อ่านทีละ 8 ช่อง

Code

```
#include <iostream>
using namespace std;
int main() {
    int x = 10;
    int* p = &x;
    cout << x << endl;
    cout << *p << endl;
    return 0;
}
```

คำถาม



1. โปรแกรมนี้แสดงผลอะไร
2. เพราะเหตุใด *p ถึงให้ค่าเดียวกับ x

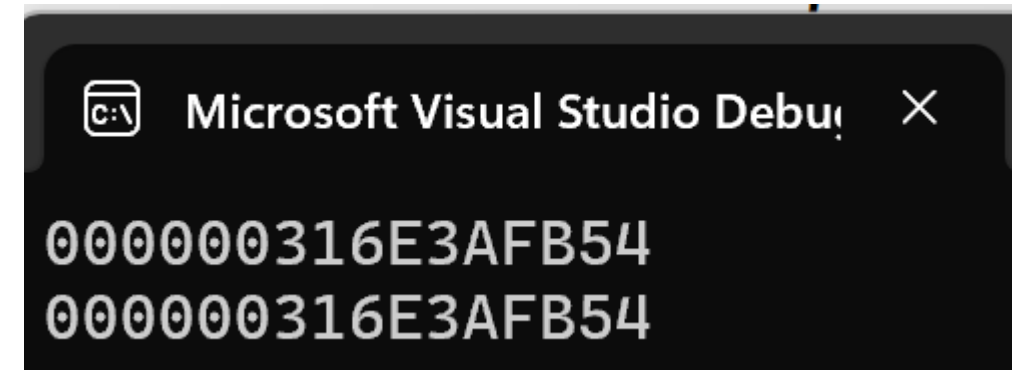
Code

```
#include <iostream>
using namespace std;
int main() {
    int score = 50;
    int* ptr = &score;

    cout << &score << endl;
    cout << ptr << endl;

    return 0;
}
```

คำถาม

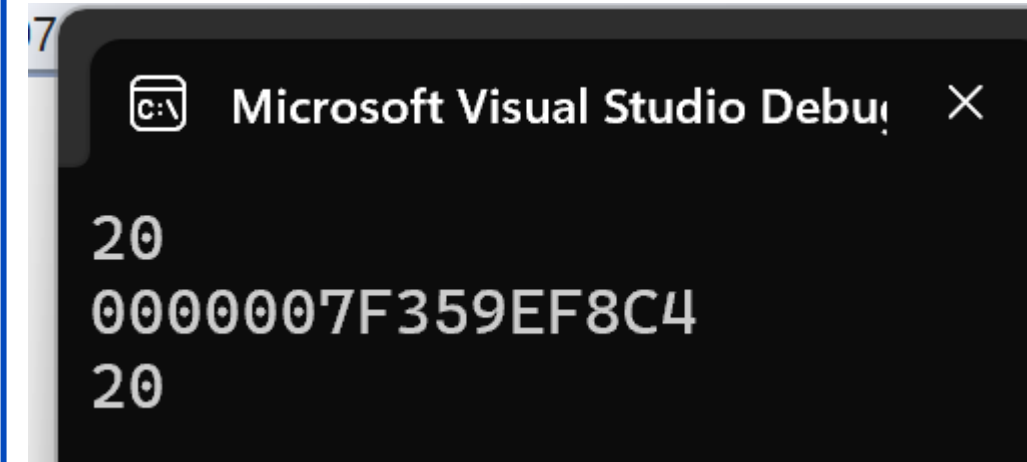


ค่าที่แสดงออกมาต่างกันหรือไม่
สองบรรทัดนี้เกี่ยวข้องกันอย่างไร

Code

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int* p = &a;
    *p = 20;
    cout << a << endl;
    cout << p << endl;
    cout << *p << endl;
    return 0;
}
```

คำถาม



โปรแกรมแสดงค่าอะไร

ถ้าแก้ *p = 30; จะเกิดอะไรขึ้น

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int* p = &a;
    *p = 20; // เปลี่ยนค่าของ a ผ่าน pointer
    cout << "Value of a          = " << a << endl;
    cout << "Address of a        = " << &a << endl;
    cout << "Value stored in p      = " << p << endl;
    cout << "Value pointed by *p     = " << *p << endl;
    cout << "Address of pointer p    = " << &p << endl;

    return 0;
}
```

Microsoft Visual Studio Debug Console

Value of a	= 20
Address of a	= 000000DBF273F614
Value stored in p	= 000000DBF273F614
Value pointed by *p	= 20
Address of pointer p	= 000000DBF273F638

```
#include <iostream>
using namespace std;

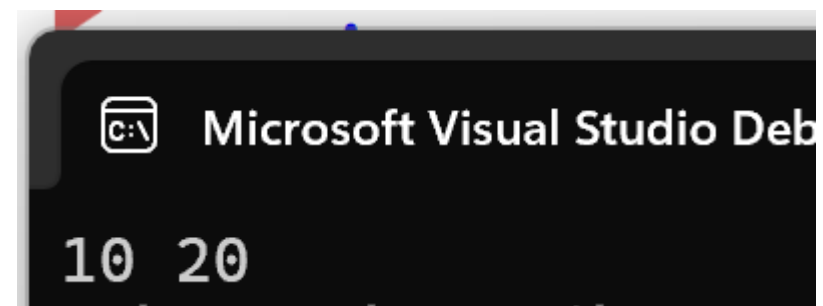
int main() {
    int x = 1, y = 2;
    int* p;

    p = &x;
    *p = 10;

    p = &y;
    *p = 20;

    cout << x << " " << y;

    return 0;
}
```



```
#include <iostream>
using namespace std;

int main() {
    double d = 3.14;
    int* p = &d;    //
    cout << *p << endl;

    return 0;
}
```

บรรทัดนี้ผิดเพราะอะไร

ควรแก้เป็นอะไร

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      double d = 3.14;
6      int* p = &d;    //
7      cout << *p << endl;
8
9      return 0;
10 }
11 // This code will cause a compilation error because we are
```

Error List

Entire Solution

1 Error

0 Warnings

0 Messages

Build + IntelliSense

Code

Description

E0144

a value of type "double *" cannot be used to initialize an entity of type "int *"

```
#include <iostream>
#include <typeinfo> // ใช้บอกชนิดข้อมูล
using namespace std;

int main() {

    int    i = 10;
    double d = 3.14;
    char   c = 'A';
    float  f = 2.5f;
    bool   b = true;

    int* pi = &i;
    double* pd = &d;
    char* pc = &c;
    float* pf = &f;
    bool* pb = &b;

    return 0;
}
```



```
cout << "\n==== ADDRESS =====" << endl;
cout << "Address of i = " << &i << endl;
cout << "Address of d = " << &d << endl;
cout << "Address of c = " << (void*)&c << endl;
cout << "Address of f = " << &f << endl;
cout << "Address of b = " << &b << endl;

cout << "\n==== POINTER VALUE =====" << endl;
cout << "pi = " << pi << endl;
cout << "pd = " << pd << endl;
cout << "pc = " << (void*)pc << endl;
cout << "pf = " << pf << endl;
cout << "pb = " << pb << endl;
```

```
==== ADDRESS =====
Address of i = 00000051BA3BF744
Address of d = 00000051BA3BF768
Address of c = 00000051BA3BF784
Address of f = 00000051BA3BF7A4
Address of b = 00000051BA3BF7C4
```

```
==== POINTER VALUE =====
pi = 00000051BA3BF744
pd = 00000051BA3BF768
pc = 00000051BA3BF784
pf = 00000051BA3BF7A4
pb = 00000051BA3BF7C4
```



```
cout << "\n==== VALUE BY POINTER =====> endl;  
cout << "*pi = " << *pi << endl;  
cout << "*pd = " << *pd << endl;  
cout << "*pc = " << *pc << endl;  
cout << "*pf = " << *pf << endl;  
cout << "*pb = " << *pb << endl;
```

```
==== VALUE BY POINTER =====  
*pi = 10  
*pd = 3.14  
*pc = A  
*pf = 2.5  
*pb = 1
```

```
cout << "==== VARIABLE SIZE & TYPE =====" << endl;
cout << "i | type: " << typeid(i).name()
    << " | sizeof: " << sizeof(i) << " bytes" << endl;

cout << "d | type: " << typeid(d).name()
    << " | sizeof: " << sizeof(d) << " bytes" << endl;

cout << "c | type: " << typeid(c).name()
    << " | sizeof: " << sizeof(c) << " byte" << endl;

cout << "f | type: " << typeid(f).name()
    << " | sizeof: " << sizeof(f) << " bytes" << endl;

cout << "b | type: " << typeid(b).name()
    << " | sizeof: " << sizeof(b) << " byte" << endl;
```

```
==== VARIABLE SIZE & TYPE =====
i | type: int | sizeof: 4 bytes
d | type: double | sizeof: 8 bytes
c | type: char | sizeof: 1 byte
f | type: float | sizeof: 4 bytes
b | type: bool | sizeof: 1 byte
```

```
cout << "\n==== POINTER SIZE & TYPE =====" << endl;
cout << "pi | type: " << typeid(pi).name()
    << " | sizeof: " << sizeof(pi) << " bytes" << endl;

cout << "pd | type: " << typeid(pd).name()
    << " | sizeof: " << sizeof(pd) << " bytes" << endl;

cout << "pc | type: " << typeid(pc).name()
    << " | sizeof: " << sizeof(pc) << " bytes" << endl;

cout << "pf | type: " << typeid(pf).name()
    << " | sizeof: " << sizeof(pf) << " bytes" << endl;

cout << "pb | type: " << typeid(pb).name()
    << " | sizeof: " << sizeof(pb) << " bytes" << endl;
```

```
==== POINTER SIZE & TYPE =====
pi | type: int * __ptr64 | sizeof: 8 bytes
pd | type: double * __ptr64 | sizeof: 8 bytes
pc | type: char * __ptr64 | sizeof: 8 bytes
pf | type: float * __ptr64 | sizeof: 8 bytes
pb | type: bool * __ptr64 | sizeof: 8 bytes
```

ระบบ	ขนาด pointer
32-bit	4 bytes
64-bit	8 bytes

ระบบ	ขนาด pointer
32-bit	4 bytes
64-bit	8 bytes

```
===== POINTER SIZE & TYPE =====
pi | type: int * __ptr64 | sizeof: 8 bytes
pd | type: double * __ptr64 | sizeof: 8 bytes
pc | type: char * __ptr64 | sizeof: 8 bytes
pf | type: float * __ptr64 | sizeof: 8 bytes
pb | type: bool * __ptr64 | sizeof: 8 bytes
```

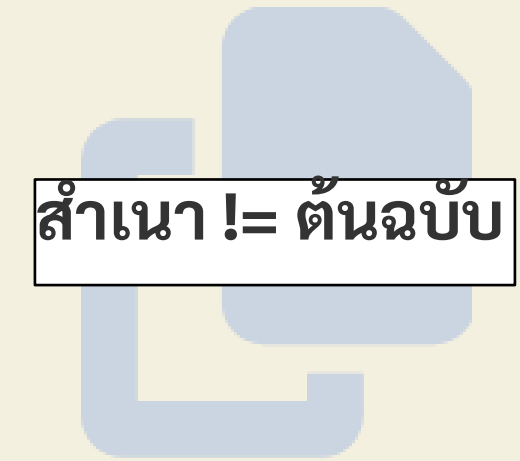
- ◇ pointer เก็บแค่ “address”
- ◇ address มีความยาวตายตัวตามระบบ
- ◇ ไม่สนใจว่าชี้ int, double, char

6. ทำไมต้องใช้ Pointer? (ปัญหา Pass by Value)

แบบปกติ (Pass by Value)

เหมือนการ "ถ่ายเอกสาร" ส่ง给朋友 เพื่อนขีดเขียนลงบนสำเนา
ต้นฉบับของเราไม่เปลี่ยน

```
void tryChange(int n) {  
    n = 999; // เปลี่ยนแค่ตัวสำเนาในนี้  
}  
  
int x = 10;  
tryChange(x);  
cout << x; // ผลลัพธ์ยังคงเป็น 10!
```



สำเนา != ต้นฉบับ

7. ทางแก้: Pass by Reference

ใช้ Pointer (Pass by Reference)

เหมือนการ "ยืมกุญแจห้อง" ให้เพื่อน เพื่อนไขเข้าไปเปลี่ยนของในห้องเราได้จริง



```
void realChange(int* ptr) {  
    *ptr = 999; // เข้าไปแก้ที่ address จริง  
}  
  
int x = 10;  
realChange(&x); // ส่งกุญแจ (Address) ไป  
cout << x; // ผลลัพธ์เปลี่ยนเป็น 999!
```


Code Review

Analyzing Functions A and B



Function A แบบ pointer

```
void realChange(int* ptr) {  
    *ptr = 999;  
}
```

Function B แบบ reference

```
void realChange(int& ptr) {  
    ptr = 999;  
}
```

```
int x = 10;  
realChange(&x); // แบบ pointer  
realChange(x); // แบบ reference  
  
cout << x;    // 999
```

มุมมอง Memory

◇ Pointer

```
int x = 10;  
int* p = &x;  
Memory:  
p → x → 10
```

◇ Reference

```
int x = 10;  
int& r = x;  
Memory:  
r ≡ x → 10
```

☞ Reference ไม่มีตัวตนใหม่ใน memory
☞ เป็นแค่ “อีกชื่อหนึ่งของ x”

8. Array กับ Pointer คือเพื่อนสนิทกัน

ชื่อของ Array จริงๆ แล้วคือ Pointer ที่ชี้ไปยังสมาชิกตัวแรก (Index 0)

```
int arr[] = {10, 20, 30};
```

```
int* ptr = arr;
```

```
// สองบรรทัดนี้ได้ผลเหมือนกัน
```

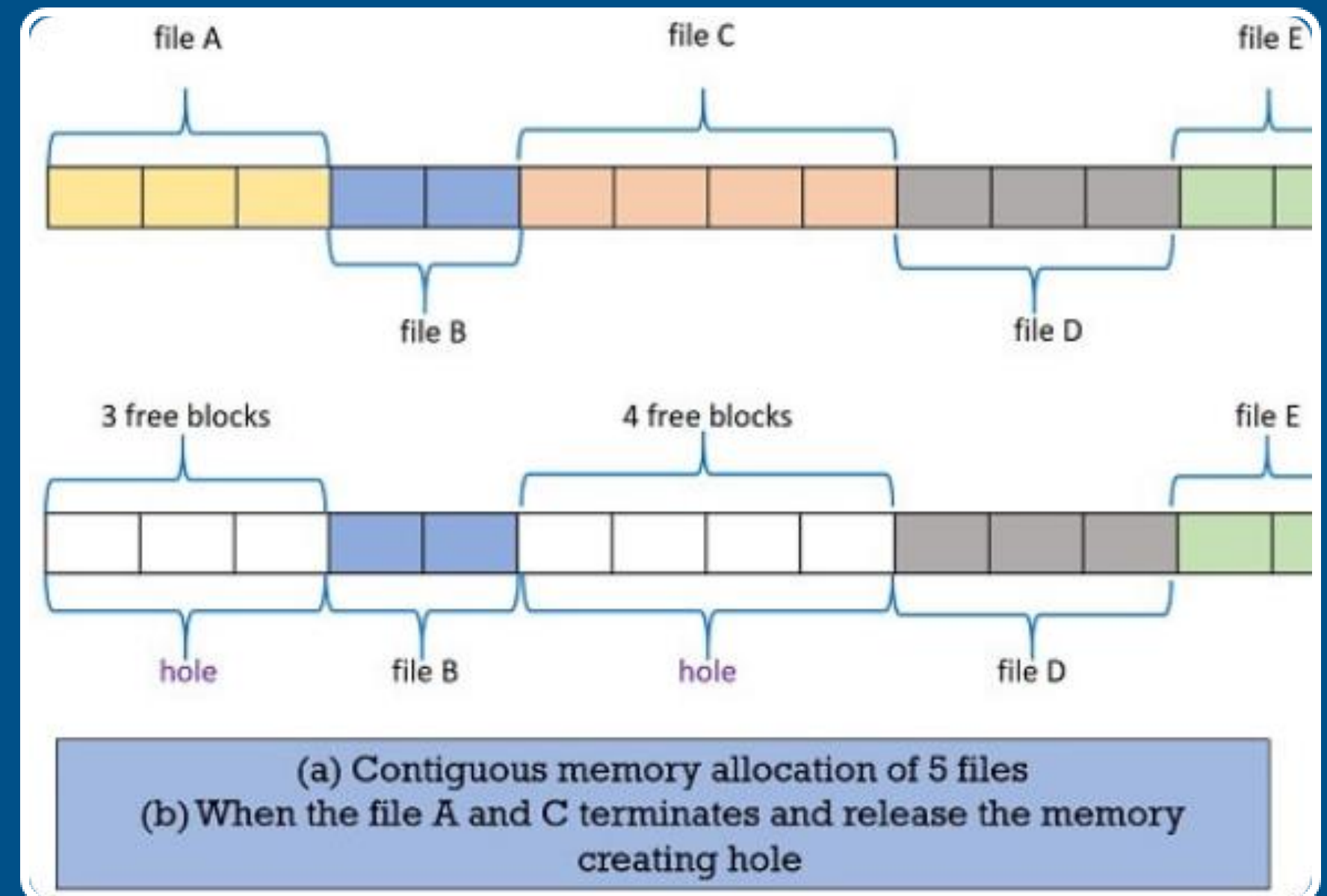
```
cout << arr[0]; // 10
```

```
cout << *ptr; // 10
```

```
// ตัวถัดไป
```

```
cout << arr[1]; // 20
```

```
cout << *(ptr + 1); // 20
```



9. คณิตศาสตร์ Pointer (Pointer Arithmetic)

การบวกเลข Pointer ไม่ได้บวกทีละ 1 หน่วย แต่บวกตาม "ขนาดของชนิดตัวแปร"

int* ptr (4 bytes)

สมมติ ptr อยู่ที่ 1000

```
ptr + 1 --> 1004  
ptr + 2 --> 1008
```

*กระโดดทีละ 4 ช่อง

char* ptr (1 byte)

สมมติ ptr อยู่ที่ 1000

```
ptr + 1 --> 1001  
ptr + 2 --> 1002
```

*กระโดดทีละ 1 ช่อง

ตัวอย่างที่ 1: Array กับ Pointer

พื้นฐาน

```
#include <iostream>
using namespace std;

int main() {

    int arr[] = { 10, 20, 30 };
    int* ptr = arr;    // ชี้ไปยังสมาชิกตัวแรก (arr[0])

    cout << "arr[0] = " << arr[0] << endl;
    cout << "*ptr    = " << *ptr << endl;

    cout << "arr[1] = " << arr[1] << endl;
    cout << "*(ptr + 1) = " << *(ptr + 1) << endl;

    cout << "arr[2] = " << arr[2] << endl;
    cout << "*(ptr + 2) = " << *(ptr + 2) << endl;

    return 0;
}
```

```
arr[0] = 10
*ptr    = 10
arr[1] = 20
*(ptr + 1) = 20
arr[2] = 30
*(ptr + 2) = 30
```



```
#include <iostream>
using namespace std;

int main() {

    int arr[] = { 10, 20, 30 };
    int* ptr = arr;

    cout << "Address of arr      = " << arr << endl;
    cout << "Address in ptr      = " << ptr << endl;
    cout << "Address of arr[1]    = " << &arr[1] << endl;
    cout << "ptr + 1              = " << (ptr + 1) << endl;

    return 0;
}
```

Microsoft Visual Studio Debug Console

Address of arr	=	000000672050F7C8
Address in ptr	=	000000672050F7C8
Address of arr[1]	=	000000672050F7CC
ptr + 1	=	000000672050F7CC

ตัวอย่างที่ 3.1 : `char[]` กับ **Pointer** (พื้นฐาน)

```
#include <iostream>
using namespace std;

int main() {

    char str[] = "Hello";
    char* p = str;    // ชี้ไปที่ตัวอักษรตัวแรก

    cout << "String (str)      = " << str << endl;
    cout << "String (p)       = " << p << endl;

    cout << "First char (*p)    = " << *p << endl;
    cout << "Second char     = " << *(p + 1) << endl;
    cout << "Third char      = " << *(p + 2) << endl;

    cout << "\n=== Address ===" << endl;
    cout << "Address of str      = " << (void*)str << endl;
    cout << "Address stored in p = " << (void*)p << endl;
    cout << "Address of p       = " << &p << endl;

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
String (str)      = Hello
String (p)       = Hello
First char (*p)   = H
Second char      = e
Third char       = l

=== Address ===
Address of str    = 0000000CF34FFAA4
Address stored in p = 0000000CF34FFAA4
Address of p     = 0000000CF34FFAC8
```


ตัวอย่างที่ 3.2 : เดิน String ด้วย Pointer (loop)

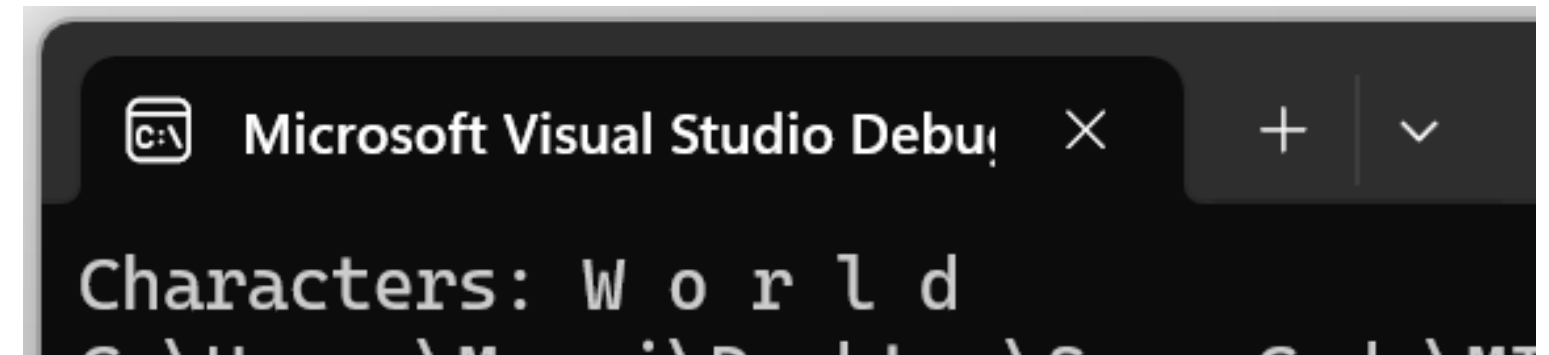
```
#include <iostream>
using namespace std;

int main() {

    char str[] = "World";
    char* p = str;

    cout << "Characters: ";
    while (*p != '\0') {
        cout << *p << " ";
        p++;
    }

    return 0;
}
```



ตัวอย่างที่ 4 : พฤติกรรมการแสดงผลของ Pointer แต่ละชนิด

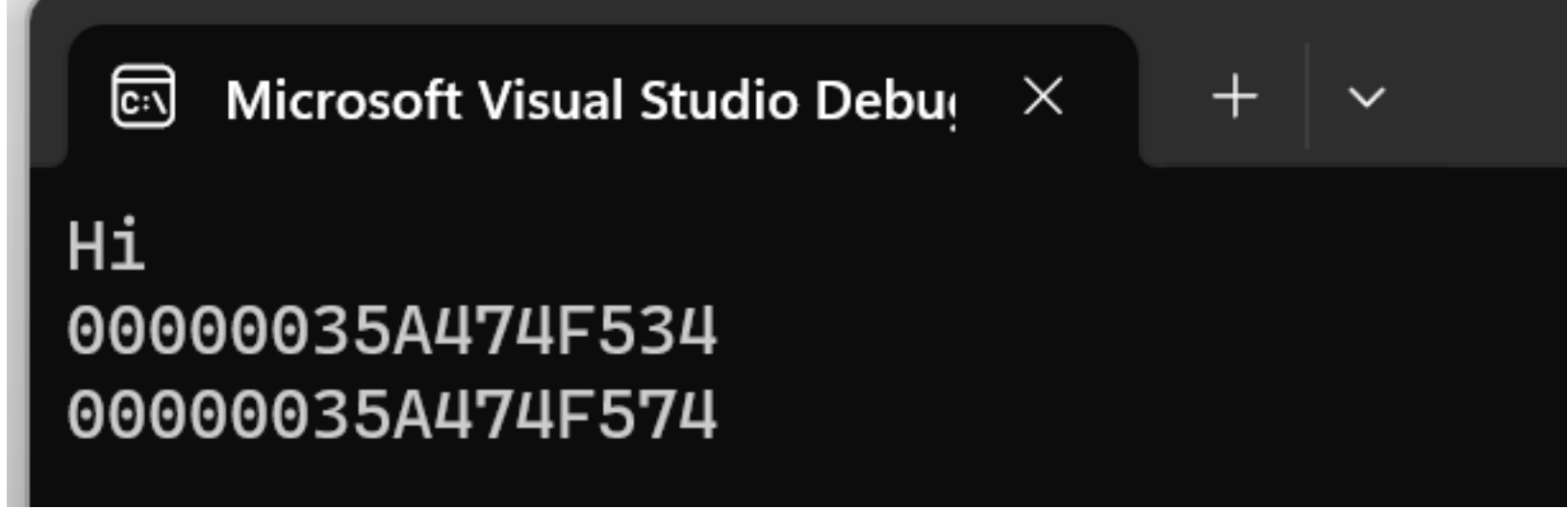
```
#include <iostream>
using namespace std;

int main() {

    char str[] = "Hi";
    char* cp = str;
    int x = 5;
    int* ip = &x;

    cout << cp << endl;           // Hi
    cout << (void*)cp << endl;     // address
    cout << ip << endl;           // address

    return 0;
}
```



```
Microsoft Visual Studio Debug Console
Hi
00000035A474F534
00000035A474F574
```

ตัวอย่างที่ 4 : พฤติกรรมการแสดงผลของ
Pointer แต่ละชนิด

Pointer	cout << p แสดง
char*	string
const char*	string
int*	address
double*	address
void*	address

ตัวอย่างที่ 5 : Array กับ Pointer (หลายชนิดข้อมูล)

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    // ===== char[] =====
    char cArr[] = "Hi";
    char* cp = cArr;

    // ===== int[] =====
    int iArr[] = { 10, 20, 30 };
    int* ip = iArr;

    // ===== string[] =====
    string sArr[] = { "Hello", "World" };
    string* sp = sArr;

    cout << "===== char[] =====" << endl;
    cout << "cArr" << " = " << cArr << endl;           // string
    cout << "cp" << " = " << cp << endl;                 // string
    cout << "*cp" << " = " << *cp << endl;                 // 'H'
    cout << "Address cArr" << " = " << (void*)cArr << endl;
    cout << "cp + 1" << " = " << (void*)(cp + 1) << endl;

    cout << "\n===== int[] =====" << endl;
    cout << "iArr[0]" << " = " << iArr[0] << endl;
    cout << "*ip" << " = " << *ip << endl;
    cout << "iArr[1]" << " = " << iArr[1] << endl;
    cout << "*(ip + 1)" << " = " << *(ip + 1) << endl;
    cout << "Address iArr" << " = " << ip << endl;
    cout << "ip + 1" << " = " << (ip + 1) << endl;

    cout << "\n===== string[] =====" << endl;
    cout << "sArr[0]" << " = " << sArr[0] << endl;
    cout << "*sp" << " = " << *sp << endl;
    cout << "sArr[1]" << " = " << sArr[1] << endl;
    cout << "*(sp + 1)" << " = " << *(sp + 1) << endl;
    cout << "Address sArr" << " = " << sp << endl;
    cout << "sp + 1" << " = " << (sp + 1) << endl;
}
```

```
return 0;
```

```
Microsoft Visual Studio Debug Console

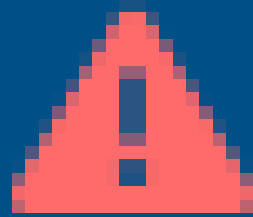
===== char[] =====
cArr          = Hi
cp            = Hi
*cp           = H
Address cArr   = 000000888DDEF5B4
cp + 1        = 000000888DDEF5B5

===== int[] =====
iArr[0]       = 10
*ip           = 10
iArr[1]       = 20
*(ip + 1)     = 20
Address iArr   = 000000888DDEF5F8
ip + 1        = 000000888DDEF5FC

===== string[] =====
sArr[0]       = Hello
*sp           = Hello
sArr[1]       = World
*(sp + 1)     = World
Address sArr   = 000000888DDEF650
sp + 1        = 000000888DDEF678
```

10. Dynamic Memory (Stack vs Heap)

เมื่อเราไม่รู้ขนาดข้อมูลล่วงหน้า เราต้องจองหน่วยความจำเอง (Heap) โดยใช้คำสั่ง new



ถ้าลืม delete จะเกิด Memory Leak (แรมรั่ว)

```
// 1. จองพื้นที่ (new)
```

```
int* ptr = new int;
```

```
*ptr = 500;
```

```
// 2. ใช้งาน...
```

```
cout << *ptr;
```

```
// 3. คืนพื้นที่ (delete) -> สำคัญมาก!
```

```
delete ptr;
```


ตัวอย่างที่ 1 : Stack Memory (จอง อัตโนมัติ)

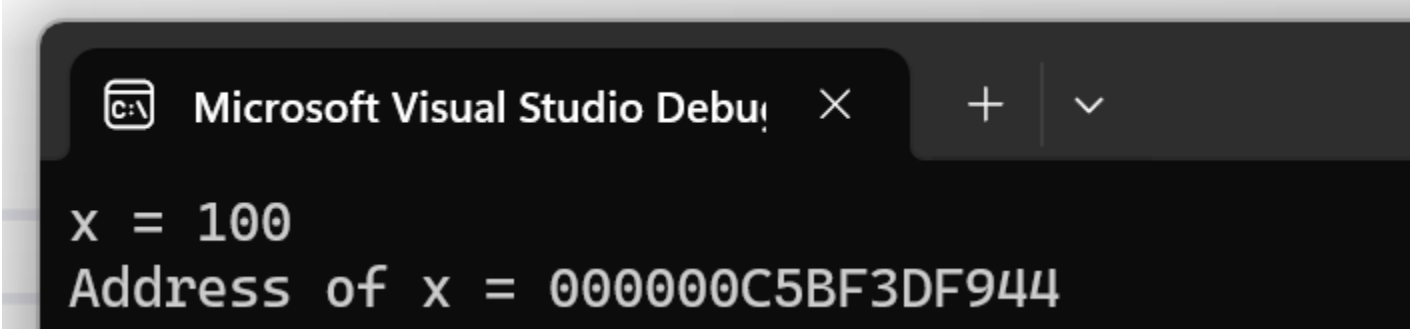
```
#include <iostream>
using namespace std;

int main() {

    int x = 100;    // อยู่ใน Stack

    cout << "x = " << x << endl;
    cout << "Address of x = " << &x << endl;

    return 0;      // ออกจาก main → คืนพื้นที่อัตโนมัติ
}
```



Microsoft Visual Studio Debug Console output:

```
x = 100
Address of x = 000000C5BF3DF944
```

ตัวอย่างที่ 2 : Heap Memory (new / delete)

```
#include <iostream>
using namespace std;

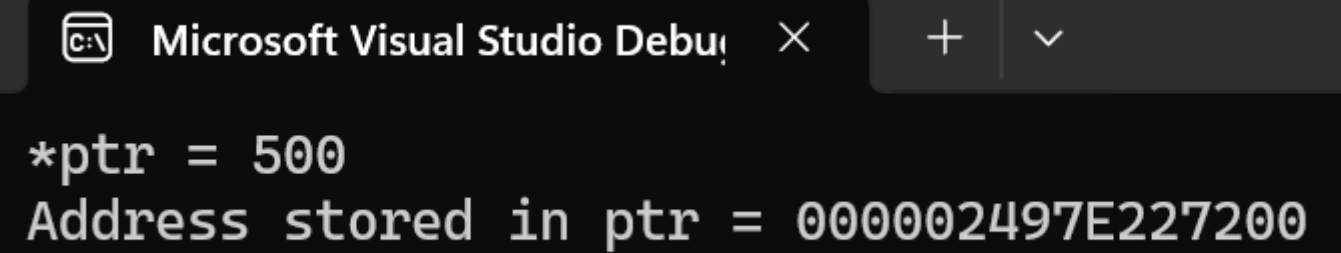
int main() {

    int* ptr = new int;    // จองหน่วยความจำบน Heap
    *ptr = 500;

    cout << "*ptr = " << *ptr << endl;
    cout << "Address stored in ptr = " << ptr << endl;

    delete ptr;           // คืนหน่วยความจำ (สำคัญ!)
    ptr = nullptr;        // ป้องกัน dangling pointer

    return 0;
}
```



Microsoft Visual Studio Debug Console

*ptr = 500
Address stored in ptr = 000002497E227200

ตัวอย่างที่ 10.3 : เปรียบเทียบ **Stack vs Heap**

```
#include <iostream>
using namespace std;

int main() {

    int a = 10;           // Stack
    int* b = new int;     // Heap
    *b = 20;

    cout << "Stack a = " << a << endl;
    cout << "Heap *b = " << *b << endl;

    delete b;

    return 0;
}
```

Microsoft Visual Studio Debug Console

```
Stack a = 10
Heap *b = 20
```

ตัวอย่างที่ 4 : Heap Array (new[] / delete[])

```
#include <iostream>
using namespace std;

int main() {

    int* arr = new int[3];    // จอง array บน Heap

    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    for (int i = 0; i < 3; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr;            // ต้องใช้ delete[]

    return 0;
}
```

Microsoft Visual Studio Debug Console

10 20 30

11. ข้อควรระวัง (Common Pitfalls)



Uninitialized Pointer

การประกาศ `int* p;` โดยไม่
กำหนดค่าเริ่มต้น มันอาจชี้ไปที่มั่วๆ
ทำให้โปรแกรมพัง (Crash) ได้



Dangling Pointer

Pointer ที่ชี้ไปยังตำแหน่งที่ถูก
delete ไปแล้ว แต่ยังพยายามเข้าไป
ใช้งาน



วิธีป้องกัน

ควรกำหนดค่าเริ่มต้นเป็น `nullptr`
เสมอถ้ายังไม่มีที่ชี้
`int* p = nullptr;`

11. ข้อควรระวัง (Common Pitfalls)

Program A : แสดง Address

```
int x = 100;  
cout << "Address of x = " << &x << endl;  
cout << "Value of x = " << x << endl;  
cin.get(); // หยุดโปรแกรมไว้ดู address
```

Program B : พยายามใช้ Address จาก Program A

```
int* p = (int*)0x61ff10;  
// address จาก Program A (สมมติ)  
cout << *p << endl;    // ✗
```



วิธีป้องกัน

1. แต่ละโปรแกรม = คนละ **Process**

- 1) OS ให้ **Virtual Memory** แยกกัน
- 2) Address 0x61ff10 ใน Program A
≠ Address 0x61ff10 ใน Program B

2. มีระบบป้องกันชื่อว่า **ASLR**

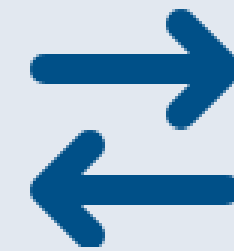
- 1) Address Space Layout Randomization
- 2) ทุกครั้งที่รัน → address จะสุ่มใหม่
- 3) ป้องกันการโจมตี (เช่น buffer overflow)

12. ตัวอย่างยอดนิยม: ฟังก์ชันสลับค่า (Swap)

โจทย์: เขียนฟังก์ชันเพื่อสลับค่าตัวเลข 2 ตัว

```
void swap(int* a, int* b) {  
    int temp = *a; // เก็บค่า a ไว้  
    *a = *b;      // เอาค่า b ใส่ a  
    *b = temp;    // เอาค่าเก่า a ใส่ b  
}
```

```
int main() {  
    int x = 5, y = 10;  
    swap(&x, &y);  
    // ตอนนี้ x = 10, y = 5  
}
```



ถ้าไม่ใช่ Pointer เราจะสลับค่าได้แค่ในฟังก์ชัน แต่ตัวแปร
จริงข้างนอกจะไม่เปลี่ยน

ตัวอย่างที่ 1 : เรียง Array แบบพื้นฐาน (Bubble Sort)

```
#include <iostream>
using namespace std;

// ฟังก์ชันแสดง Array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

// ฟังก์ชันสลับค่า
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// ฟังก์ชันเรียงจากน้อยไปมาก
void sortAscending(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {

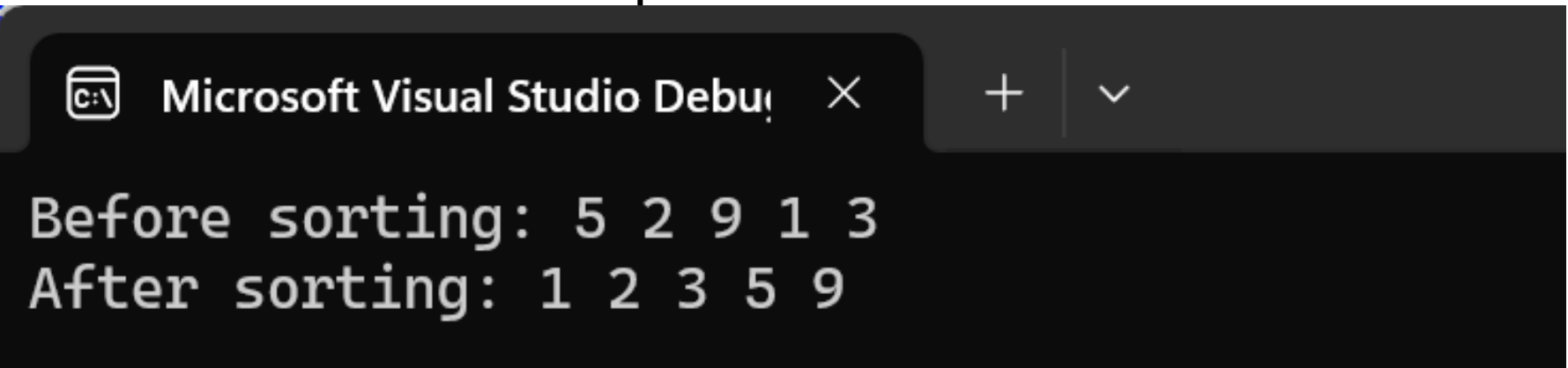
    int arr[] = { 5, 2, 9, 1, 3 };
    int size = 5;

    cout << "Before sorting: ";
    printArray(arr, size);

    sortAscending(arr, size);

    cout << "After sorting: ";
    printArray(arr, size);

    return 0;
}
```



ตัวอย่างที่ 1 : เรียง Array แบบพื้นฐาน (Bubble Sort)

```
#include <iostream>
using namespace std;

int main() {

    int arr[] = { 5, 2, 9, 1, 3 };
    int size = 5;

    cout << "Before sorting: ";
    printArray(arr, size);

    sortAscending(arr, size);

    cout << "After sorting: ";
    printArray(arr, size);

    return 0;
}
```

```
// ฟังก์ชันแสดง Array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
}

// ฟังก์ชันสลับค่า
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

// ฟังก์ชันเรียงจากน้อยไปมาก
void sortAscending(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```



```
#include <iostream>
using namespace std;

int main() {

    int arr[] = { 5, 2, 9, 1, 3 };
    int size = 5;

    cout << "Before sorting: ";
    printArray(arr, size);

    sortAscending(arr, size);

    cout << "After sorting: ";
    printArray(arr, size);

    return 0;
}
```

```
// แสดง Array ด้วย pointer
void printArray(int* p, int size) {
    for (int i = 0; i < size; i++) {
        cout << *(p + i) << " ";
    }

    cout << endl;
}

// สลับค่าโดยใช้ pointer
void swapValue(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// เรียงจากน้อยไปมาก (Bubble Sort + Pointer)
void sortAscending(int* p, int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - 1 - i; j++) {
            if (*(p + j) > *(p + j + 1)) {
                swapValue(p + j, p + j + 1);
            }
        }
    }
}
```

สรุปสิ่งที่ต้องจำ (Key Takeaways)

- **Pointer** คือตัวแปรที่เก็บ Address (บ้านเลขที่)
- **& (Ampersand)** ใช้หาตำแหน่งที่อยู่
- *** (Asterisk)** ใช้เข้าถึงค่าข้อมูลในตำแหน่งนั้น
- ใช้ **Pass by Reference** เพื่อแก้ไขค่าจริงในฟังก์ชัน
- ระวังเรื่อง **Memory Leak** เมื่อใช้ new ต้องคู่กับ delete เสมอ



Sutit Ongart
Sutit@mut.ac.th
sutit.ongart@gmail.com

END